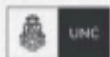


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el alumno

Introducción a Devops



**WE WANT
SKILLS!**

mundosE

Manual para el Alumno: Introducción a DevOps	2
Objetivo del encuentro:	2
Contenido a desarrollar:	2
1. Fundamentos de DevOps	3
La Cultura DevOps	3
Recursos para profundizar:	3
Automatización, Integración y Colaboración	4
Automatización:	4
Integración:	4
Colaboración:	4
Recursos para profundizar:	4
¿Qué es DevOps? Definición y Propósito	5
Beneficios principales:	5
Recursos para profundizar:	5
¿Por qué es importante DevOps?	6
Recursos adicionales:	6
2. Comparación de DevOps con otras Metodologías (Agile y Waterfall)	7
DevOps vs. Metodologías Tradicionales	7
Recursos para profundizar:	7
3. Lean Thinking y Value Stream Mapping (VSM)	8
Conceptos clave de Lean Thinking	8
Principios de Lean Thinking:	8
¿Qué es el Value Stream Mapping (VSM)?	8
Recursos para aprender más:	9
4. Métricas en DevOps	10
Principales métricas en DevOps	10
1. Frecuencia de Despliegue:	10
2. Tiempo de Ciclo (Lead Time):	10
3. MTTR (Mean Time to Recovery):	10
4. Tasa de Fallos en Cambios:	10
Bibliografía	11

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En él encontraremos un resumen de cada tema tratado en el encuentro, así como también recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el Alumno: Introducción a Devops

Objetivo del encuentro:

Brindar una comprensión clara de qué es DevOps, su propósito y la importancia de la colaboración entre los equipos de desarrollo y operaciones para optimizar el ciclo de vida del software. Los estudiantes comprenderán cómo DevOps reduce los tiempos de entrega y mejora la calidad del software a través de la automatización y la integración continua.

Contenido a desarrollar:

- ✓ Definición y propósito de DevOps
- ✓ La importancia de la cultura DevOps
- ✓ Comparación de DevOps con metodologías tradicionales como Agile y Waterfall
- ✓ Beneficios de la adopción de DevOps
- ✓ Principios clave: automatización, integración y colaboración

1. Fundamentos de DevOps

La Cultura DevOps

La cultura DevOps se basa en la colaboración, la confianza mutua y la responsabilidad compartida entre los equipos de desarrollo y operaciones. Fomenta un ambiente en el que todos los miembros trabajan juntos para lograr 2 objetivos principales, entregas más frecuentes y más confiables (que menor cantidad de errores).

Algunos de los principios de la cultura DevOps:

- Comunicación abierta y frecuente entre equipos.
- Colaboración en todas las etapas del desarrollo.
- Responsabilidad compartida por el éxito del producto.

En la cultura de DevOps hay muchas ceremonias que son de uso común y que fomentan y dan forma a esta forma de trabajo, entre ellas podemos observar:

- Reuniones regulares entre equipos de desarrollo y operaciones.
- Implementación de feedback continuo para la mejora.

Un ejemplo práctico podría ser:

Imagina que un equipo de desarrollo entrega código al equipo de operaciones para implementar en producción. En lugar de realizar este proceso de manera aislada, ambos equipos trabajan juntos para resolver problemas y garantizar una entrega fluida. Ya vamos a ir viendo que esto no siempre fue así y que gracias a DevOps cada vez es más frecuente aprovechar de las ventajas de estas interacciones.

Recursos para profundizar:

Video: [DevOps en 5 minutos | ¿Qué es DevOps? | Explicación DevOps](#)

Fuente: Tecnología 4.0 (2023)

Blog: [¿En que consisten las DevOps?](#)

Fuente: AWS.com

Automatización, Integración y Colaboración

DevOps se fundamenta en tres principios clave que impulsan su éxito: automatización, integración y colaboración.

Automatización:

Consiste en reemplazar tareas manuales con procesos automatizados para reducir errores y acelerar el desarrollo.

Ejemplo: Uso de herramientas como Jenkins para automatizar pruebas y despliegues.

Integración:

Asegura que los diferentes componentes del software trabajen juntos de manera fluida.

Ejemplo: Implementación de pipelines de CI/CD.

Colaboración:

Fomenta el trabajo conjunto entre equipos de desarrollo, operaciones y calidad.

Un ejemplo habitual en las organizaciones es la creación de equipos multifuncionales (desarrolladores, administradores de servidores, administradores de base de datos, administradores de seguridad, etc.) para abordar problemas en tiempo real.

Recursos para profundizar:

Video: [Automatización y CI/CD](#).

Fuente: Pelado Nerd (2020)

Video 2: [DevOps CI CD automation Meme | Knowledge Sharing](#) (meme de lo que a veces ocurre con los pipelines, lo que debemos evitar)

Fuente: Careerya (2021)

Artículo: [Cómo fomentar la colaboración en DevOps](#).

Fuente: itmastersmag (2023)

¿Qué es DevOps? Definición y Propósito

DevOps es una filosofía de trabajo que busca integrar los equipos de desarrollo ("Dev") y operaciones ("Ops") para lograr entregas de software "más rápidas", confiables (sin errores, ni problemas de seguridad) y eficientes. Esta integración se apoya en la automatización de procesos, la colaboración continua y el uso de herramientas que facilitan estas tareas.

Para lograr esto, DevOps se basa en los siguientes pilares:

- Colaboración: Eliminar barreras entre desarrollo y operaciones.
- Automatización: Minimizar tareas manuales para reducir errores y acelerar procesos.
- Entrega continua: Implementar cambios frecuentes y confiables en producción.

Beneficios principales:

- Acelera el tiempo de entrega de software.
- Mejora la calidad mediante pruebas automatizadas.
- Incrementa la confiabilidad y escalabilidad de sistemas.

Ejemplo:

Pensá en alguna app de comercio electrónico (ML, Amazon, Tiendiamia, etc) que necesita actualizaciones frecuentes. Con DevOps, los equipos pueden lanzar cambios pequeños y constantes, asegurándose de que cada uno sea probado y desplegado automáticamente siempre con la app funcional, sin perder de esta manera ninguna transacción.

Recursos para profundizar:

Video: [¿Que es ¿Qué es y qué no es DevOps? Y que no es DevOps?](#)

Fuente: edteam (2022)

Blog: [A Brief History of DevOps and the Link to Cloud Development Environments](#) (inglés)

Fuente: devops.com (2024)

¿Por qué es importante DevOps?

DevOps resuelve los problemas tradicionales de los "silos" entre desarrollo y operaciones, donde ambos equipos trabajan de forma aislada, generando demoras y errores en la entrega y actualización de software.

Problema tradicional: Las actualizaciones lanzadas por el equipo de desarrollo suelen fallar en producción debido a la falta de comunicación y pruebas.

Solución con DevOps: Automatización de pruebas y despliegues para minimizar errores humanos y tiempos de espera.

Recursos adicionales:

Video: [DevOps y Agile: ¿Son lo mismo? ¿Cómo conviven?](#)

Fuente: Amin Espinoza (2022)

Artículo: [The business benefits of DevOps](#)

Fuente: Devops.com (2015) - inglés

2. Comparación de DevOps con otras Metodologías (Agile y Waterfall)

DevOps vs. Metodologías Tradicionales

Waterfall:

Enfoque lineal donde cada etapa (análisis, diseño, desarrollo, prueba, despliegue) se completa antes de pasar a la siguiente.

Problema: Rigidez ante cambios. Ejemplo: Una modificación en los requisitos implica rehacer etapas completas.

Agile:

Enfoque iterativo y flexible, enfocado en entregas frecuentes e incrementales de software funcional.

Problema: No aborda completamente la integración con operaciones.

DevOps:

Se construye sobre Agile al incorporar operaciones, automatización y monitorización continua.

Ejemplo: Uso de pipelines de CI/CD para asegurar despliegues constantes y fiables.

Tabla Comparativa:

Característica	Waterfall	Agile	DevOps
Enfoque	Secuencial	Iterativo	Colaborativo
Flexibilidad	Baja	Alta	Muy alta
Automatización	N/A	Limitada	Crítica
Participación Ops	Nula	Limitada	Total

Recursos para profundizar:

Video: [¿Que debería usar? ¿Scrum o Kanbam?](#)

Fuente: Sam Barrado (2024)

Artículo: [A Pro Football Approach to Agile Vs. DevOps](#)

Fuente: Devops.com (2021) - inglés

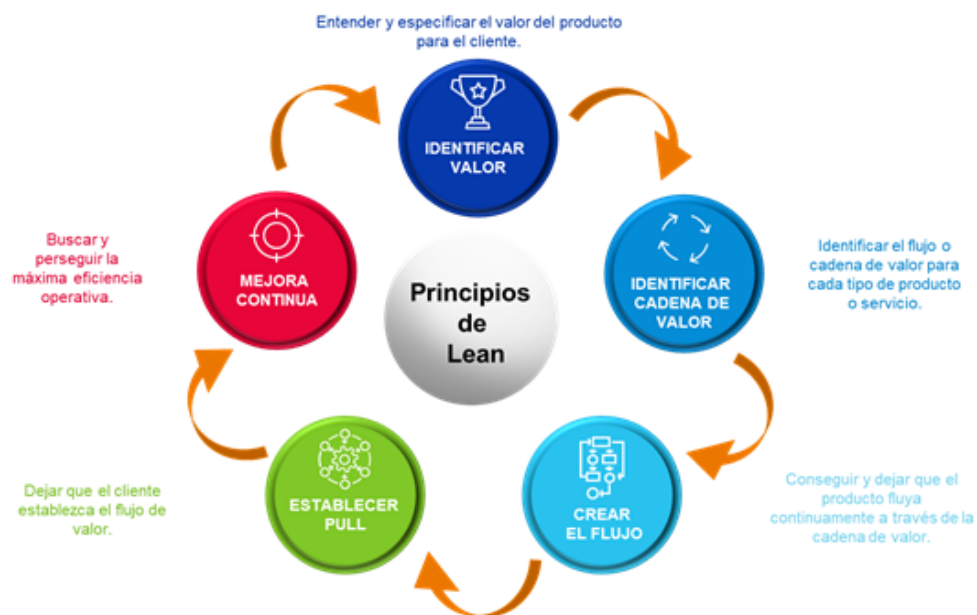
3. Lean Thinking y Value Stream Mapping (VSM)

Conceptos clave de Lean Thinking

Lean Thinking busca optimizar procesos eliminando actividades que no generan valor (desperdicios) y enfocándose en aquellas que sí lo hacen.

Principios de Lean Thinking:

- Identificar el valor para el cliente
- Mapear el flujo de valor (crear la cadena de valor)
- Crear un flujo continuo
- Establecer un sistema pull
- Buscar la mejora continua



Fuente de la imagen: <https://ayesa365.com/lean-thinking-mas-que-una-metodologia-una-filosofia-que-maximiza-el-valor-de-entrega-al-cliente/>

¿Qué es el Value Stream Mapping (VSM)?

El VSM es una herramienta utilizada para visualizar y analizar todos los pasos en un proceso, identificando cuellos de botella y desperdicios.

Cómo aplicar el VSM en DevOps:

1. **Mapeo inicial:** Identificar todas las etapas del ciclo de desarrollo y despliegue de software.
2. **Evaluación:** Marcar actividades que generan valor (por ejemplo, pruebas automatizadas) y desperdicios (por ejemplo, aprobaciones manuales repetitivas).
3. **Optimización:** Diseñar un flujo optimizado que reduzca tiempos y elimine cuellos de botella.

Ejemplo práctico:

Mapear las etapas: Codificación, pruebas, despliegue.

Identificar actividades que generan desperdicio (por ejemplo, pruebas manuales repetitivas).

Proponer mejoras como la automatización de pruebas.

Beneficios del VSM en DevOps:

- Visualización clara del proceso completo.
- Identificación de ineficiencias.
- Facilita la colaboración entre equipos para mejorar procesos.

Recursos para aprender más:

Video: [¿Qué es Lean Manufacturing? Explicado en 5 minutos](#)

Fuente: Ingeniería para to2 (2020)

Artículo: [Qué es Value Stream Mapping.](#)

Fuente: Lean.org (s/f)

4. Métricas en DevOps

Principales métricas en DevOps

1. Frecuencia de Despliegue:

Indica la rapidez con la que un equipo entrega cambios al entorno de producción.

Ejemplo: Lanzar actualizaciones diarias en vez de mensuales.

2. Tiempo de Ciclo (Lead Time):

Mide el tiempo desde que una tarea es solicitada hasta que está lista para el cliente.

Ejemplo: Reducir de semanas a días gracias a la automatización.

3. MTTR (Mean Time to Recovery):

Tiempo promedio para recuperar un sistema tras una falla.

Ejemplo: Restaurar un sistema en minutos en lugar de horas.

4. Tasa de Fallos en Cambios:

Porcentaje de cambios que requieren correcciones posteriores.

Ejemplo: Disminuir errores en despliegues con pruebas automatizadas.

Para seguir aprendiendo:

Video: [¿Cómo medir el éxito de DEVOPS a través de las FOUR KEY METRICS?](#)

Fuente: Sentries (2021)

Artículo: [Métricas DORA: ¿qué son, ventajas y cómo adoptarlas en tu empresa?](#)

Fuente: sydle.com (2023)

Bibliografía

Kim, Gene; Debois, Patrick; Humble, Jez; Willis, John. (2018). Manual de DevOps: **Cómo crear agilidad, confiabilidad y seguridad de clase mundial en organizaciones tecnológicas.** Ediciones IT. ISBN: 978-1942788379.

Humble, Jez; Farley, David. (2012). **Entrega Continua: Publicación confiable de software mediante automatización de pruebas y despliegue.** Editorial Addison-Wesley. ISBN: 978-0321601919.

Forsgren, Nicole; Humble, Jez; Kim, Gene. (2019). Acelerar: **La ciencia de Lean Software y DevOps.** Ediciones IT. ISBN: 978-1942788409.

Swartout, Phil. (2020). **DevOps: Guía Completa en Español.** Publicación Independiente. ISBN: 978-1097364261.